Лекція №2 (2 години). HTML5: Семантика та структура документа.

План лекції

1. Анатомія документа та базовий синтаксис
2. Глобальні елементи та форматування тексту
3. Семантичний HTML5: Чому він такий важливий?
4. Доступність (Web Accessibility / a11y)

Перелік умовних скорочень

- **HTML (HyperText Markup Language)** — мова гіпертекстової розмітки.
- **CSS (Cascading Style Sheets)** — каскадні таблиці стилів.
- **SEO (Search Engine Optimization)** — пошукова оптимізація.
- **a11y (Accessibility)** — веб-доступність (інклюзивність у веб-середовищі).
- **W3C (World Wide Web Consortium)** — консорціум Всесвітньої павутини, який створює веб-стандарти.
- **WCAG (Web Content Accessibility Guidelines)** — керівництво з доступності веб-контенту.
- **WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications)** — ініціатива веб-доступності для складних інтернет-застосунків.

Вступ

Гіпертекстова розмітка — це абсолютна основа будь-якої веб-сторінки. Навіть найтехнологічніші та найскладніші веб-додатки (написані на React, Vue чи Angular) зрештою компілюються (або рендеряться) у звичайний HTML. Жодна стилізація (CSS) чи інтерактивність (JavaScript) не матиме сенсу без правильного фундаменту.

Історично склалося так, що HTML створювався науковцями для обміну документами, а не для створення складних візуальних інтерфейсів. Звідси походить його фокус на "структурі" (заголовки, параграфи, списки). Однак, сучасний інтернет вимагає від розмітки набагато більшого: сторінки повинні бути доступними для скрінрідерів (якими користуються люди з вадами зору), зрозумілими для пошукових роботів (SEO-оптимізація) та легкими для підтримки самими розробниками. Саме для вирішення цих завдань у специфікації HTML5 з'явилась **семантична розмітка**.

У цій лекції ми детально розберемо анатомію HTML-документа, синтаксис, атрибути, принципи побудови правильного макету за допомогою семантичних тегів та основи веб-доступності (Web Accessibility, a11y).

---

1. Анатомія документа та базовий синтаксис

Браузер (Google Chrome, Firefox, Safari) отримує від сервера не красиву сторінку, а просто довгий рядок тексту. Щоби перетворити цей текст на структуру, зрозумілу як для програми, так і для людини, використовується синтаксис тегів.

1.1 Теги, Елементи та Атрибути

HTML-теги — це слова (чи букви), взяті в кутові дужки (`<` та `>`). Вони діють як контейнери, що "обгортають" певний контент. Існує три основні складові будь-якого елемента:

1. **Відкриваючий тег:** Вказує, де починається елемент (напр., `<p>`).
2. **Вміст (Content):** Текст або інші теги всередині.
3. **Закриваючий тег:** Вказує, де елемент закінчується (напр., `</p>`). Має косу риску `/` .

`HTML-елемент` — це сукупність цих трьох частин: відкриваючого тегу, контенту та закриваючого тегу. Наприклад: `<h1>Вітаємо на сайті!</h1>`

Існують також **самозакриваючі теги** (або "порожні елементи"), які не можуть містити тексту всередині себе. Наприклад, зображення (`<img>`), розрив рядка (`<br>`), горизонтальна лінія (`<hr>`) або поле вводу (`<input>`).

Атрибути — це додаткова інформація про елемент, яка вказується **тільки у відкриваючому тезі**. Вони завжди йдуть парами "ім'я=значення" (значення обов'язково в подвійних або одинарних лапках). Приклад: `<a href="https://example.com" target="_blank" class="nav-link">Натисни мене</a>`

- Тут `href` (hypertext reference), `target` та `class` — це атрибути. Вони описують, *куди* веде посилання, *як* воно відкривається та *який* клас стилів CSS до нього прив'язати.

1.2 Структура HTML5 документа (Boilerplate)

Жодна веб-сторінка не обходиться без базового каркасу (Boilerplate):

```
<!DOCTYPE html>
<html lang="uk">
  <head>
```

```
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Моя перша сторінка</title>
  </head>
  <body>
    <!-- Видимий контент знаходиться тут -->
  </body>
</html>
```

Давайте розберемо кожен рядок:

- `<!DOCTYPE html>` — (Document Type Declaration) Застарілий рудимент, який зберігся з часів SGML. У стандарті HTML5 це просто інструкція для браузера: "Будь ласка, рендерь цю сторінку за найсучаснішими стандартами (Standards Mode), а не вмикай режим сумісності зі старими сторінками з 90-х років (Quirks Mode)".
- `<html>` — Кореневий (головний) елемент сторінки. Всі інші елементи є його нащадками. Атрибут `lang="uk"` критично важливий для скрінрідерів (вони розумітимуть, з яким акцентом читати текст) та для систем перекладу (Google Translate).
- `<head>` — "Мозок" або технічна частина сторінки. Все, що всередині `<head>`, **невидиме** для користувача (окрім `<title>`). Тут містяться інструкції для браузера, підключення CSS-стилів та мета-дані для пошукових ботів.
  - `<meta charset="UTF-8">` — Вказує браузеру кодування символів. З 99.9% ймовірністю ви будете використовувати UTF-8, який підтримує всі мови світу та популярні емодзі. Запобігає появі "кракозябр" на сторінці.
  - `<meta name="viewport" content="width=device-width, initial-scale=1.0">` — Найважливіший тег для **мобільної адаптивності**. Він каже браузерам телефонів: "Не віддаляй сторінку, як на комп'ютері, а зроби її ширину такою ж, як ширина екрану мобільного". Без цього мобільні сайти будуть мікроскопічними.
  - `<title>` — Заголовок вкладки браузера. Впливає на те, що буде написано великими синіми буквами в результатах пошуку Google.
- `<body>` — "Тіло" сторінки. Весь видимий користувачу контент (картинки, форми, тексти, відео) повинен знаходитись виключно всередині цього тегу.

---

2. Глобальні елементи та форматування тексту

Існує десятки базових тегів, які використовуються для форматування тексту. Вони були в HTML ще з версій 2.0 та 3.0:

2.1 Заголовки (Headings)

Від `<h1>` до `<h6>`. Вони формують зміст сторінки.

- **Правило SEO:** На всій сторінці має бути лише один `<h1>` (заголовок першого або найвищого рівня), що описує загальну суть сторінки статі. Далі йде послідовна ієрархія (напр. під тема `<h2/>`, а її пункти `<h3/>`). Не можна перестрибувати рівні (одразу після `<h1>` ставити `<h4>`).
- **Правило візуалізації:** Ніколи не використовуйте теги `<h2>` чи `<h3>` просто для того, щоб зробити текст *великим і жирним*. Для цього існує CSS (`font-size`). Заголовки — це виключно про структуру.

2.2 Текстові елементи та списки

- `<p>` (Paragraph) — Параграф тексту. Браузери автоматично додають відступи до та після `<p>`.
- `<a>` (Anchor) — Гіперпосилання. Основний атрибут `href="посилання"`.
  - Атрибут `target="_blank"` відкриває посилання в новій вкладці (корисно для зовнішніх лінків).
- `<img src="image.jpg" alt="Опис картинки">` — Зображення.
  - **Критичний атрибут `alt` :** Альтернативний текст. Якщо картинка не завантажилась (проблеми з мережею), виведеться цей текст. Також цей текст зачитується вголос скрінрідерами для незрячих. Тому опис має бути описовим (не "image.png", а "Рудий кіт спить на дивані"). Пустий `alt=""` дозволяється тільки якщо картинка несе суто декоративний характер, щоб скрінрідери її просто проігнорували.
- **Списки:** Бувають нумеровані `<ol>` (Ordered List) та ненумеровані (марковані крапкою) `<ul>` (Unordered List). Елемент всередині завжди позначається тегом `<li>` (List Item).

Блокові та Рядкові елементи (Block-level vs Inline):

- **Блокові (Block):** Теги типу `<div>`, `<p>`, `<h1>`, `<ul>`. За замовчуванням вони займають 100% доступної ширини батьківського елемента (розтягуються на весь екран зліва направо). Наступний елемент буде на новому рядку (наче натиснуто Enter).
- **Рядкові (Inline):** Теги типу `<span>`, `<a>`, `<strong>`, `<em>`. Займають рівно стільки місця за шириною та висотою, скільки вимагає їх вміст (текст). Вони вишиковуються горизонтально в один рядок.

До появи HTML5 і CSS Flexbox/Grid, ключовим "цеглинкою" верстки був тег `<div>` (Division) — порожній блоковий контейнер. Сайти розроблені у 2000-2010 роках часто страждали на проблему **Div-soup** ("Суп з дівів"), коли вся сторінка складалася зі структури типу `<div><div><div><p>Текст</p></div></div></div>`, що ускладнювало її читання як розробнику, так і ботам.

---

3. Семантичний HTML5: Чому він такий важливий?

В епоху HTML5 (стандарт затверджено у 2014 році) розробникам дали нові теги, які на вигляд (для браузера) працюють абсолютно ідентично тегу `<div>`, але мають "сенс" або "семантику".

Семантичний код — це розмітка, яка передає смислове значення свого вмісту машинам (пошуковим роботам, браузерам, допоміжним програмам).

Розглянемо приклад:

- `<div class="main-menu">...</div>` — Для браузера і робота Google це просто "якийсь блок" (атрибут `class` читається тільки для CSS стилів, машини на імена класів не звертають уваги).
- `<nav>...</nav>` — Браузер і роботи розуміють: "Ага, все, що всередині цього тегу — це навігаційні посилання по сайту".

3.1 Ключові Семантичні Теги HTML5

Ось основні елементи структурування (Landmark roles):

- **`<header>`**: Вступна частина (шапка) цілої сторінки сайту (містить логотип, головне меню) або шапка окремої секції (статті). Може бути скільки завгодно на сторінці (але один головний).
- **`<nav>`**: Блок спеціально для навігаційних лінків. (Найчастіше міститься всередині `<header>`). Якщо на сторінці є кілька навігацій (віджети меню внизу сторінки), найкраща практика – використовувати їх кілька разів для головних блоків посилань.
- **`<main>`**: Головний контент сторінки. Те, заради чого користувач сюди прийшов (унікальна частина сайту). **Зустрічається лише один раз на сторінці**. Не може включати теги на кшталт бокового меню чи футера сайту.
- **`<article>`**: Незалежний і автономний шматок контенту. Це пост у блозі, новина, коментар, картка товару у магазині. Основний критерій: якщо цей блок скопіювати на інший сайт (наприклад у RSS-стрічку), він не втратить свого значення (наприклад, має свій заголовок всередині).
- **`<section>`**: Блок, що логічно поєднує тематичний контент, зазвичай з власним заголовком (наприклад "Наші послуги", "Контакти", або "Розділ 1" у великій статті). Більш загальний, ніж `<article>`. Відрізняється від звичайного `<div>` тим, що в нього є тематичний сенс, який часто іменують заголовком `<h2>` або `<h3>` всередині.
- **`<aside>`**: Контент, який пов'язаний з основним, але може бути вилучений без шкоди для розуміння сторінки. Найчастіше: бокова панель (Sidebar), рекламні блоки, блок "Схожі статті". Скрінрідери часто дозволяють перестрибнути `<aside>`, фокусуючись на `<main>`.
- **`<footer>`**: Підвал (хвіст) всієї сторінки, або підвал певної секції/статті. Зазвичай містить інформацію про автора, копірайтинг (`© 2024`), контактні дані та другорядні

посилання сайту.

3.2 Семантичний акцент у тексті

- `<strong>` (замість старого `<b>`): Важливий текст. Браузер робить його жирним, а скрінрідери читають з підсиленою інтонацією.
- `<em>` (замість `<i>`): Емоційний акцент, часто відображається курсивом. Приклад: `Вона була <em/>дуже</em> обурена`.
- `<time>`: Дозволяє явно вказати дату і час для машинного читання. Наприклад: `<time datetime="2024-03-15">15 Березня</time>`. Календарі на телефоні можуть автоматично розпізнати подію з такого тегу (створити "Івент").
- `<figure>` та `<figcaption>`: Контейнер для зображення (або діаграми, інфографіки) і його підпису. Робить зв'язок між зображенням і його описом змістовним.

3.3 Чому це вигідно бізнесу?

Робити "Семантично" часто складніше, ніж просто накидати стопку "дівів". Але у реальному e-commerce чи медіа проектах це приносить гроші:

- **SEO (Search Engine Optimization):** Робот Google (Googlebot) читає код, щоб зрозуміти, про що сайт. Коли бот бачить `<article>` з вкладеним `<h1>`, він розуміє, що це найважливіша стаття. Коли ж бот шукає дату публікації і бачить `<time>` - він точно виведе її в результатах пошуку (сніппетах). Це значно підвищує шанси сайту зайняти 1-ші позиції в Google і принести більше трафіку.
- **Доступність (Accessibility):** Опція "Reader Mode" (Режим читання) в браузерах Safari (iOS) або Firefox відкидає рекламу та стилі і показує лише чистий текст, спираючись на факт наявності `<article>`, `<main>`, `<p>`, `<h1>`. Якщо сайт був «супом дівів», режим читання у користувача не спрацює.

---

4. Доступність (Web Accessibility / a11y)

Web Accessibility (часто скорочують як **a11y** — "a" + 11 літер до + "y") — це практика створення 웹i-сайтів і додатків, якими можуть користуватись люди з обмеженими можливостями (зір, моторика рук, слух, когнітивні розлади). Веб Консорціум (W3C) розробив стандарт **WCAG (Web Content Accessibility Guidelines)**. Багато державних та банківських сайтів у Європі та США на законодавчому рівні (Акт ADA) зобов'язані відповідати цьому стандарту (як мінімум рівню AA), інакше їх оштрафують на мільйони доларів.

Основні принципи a11y, які веб-розробники зобов'язані вбудовувати у свою роботу ще на стадії написання HTML-каркасу:

1. **Навігація з Клавіатури:** Дехто не користується мишкою з фізіологічних причин. Увесь інтерактивний сайт має управлятись клавішею **Tab** (перемикання фокусу), **Space/Enter** (взаємодія) та стрілками.
   - Семантичні теги `<button>` (кнопка) або `<a>` (посилання) автоматично ловлять фокус з клавіатури, на відміну від кастомної `<div class="btn">`.
   - Атрибут `tabindex`:
     - `tabindex="0"` — додає елемент у чергу вкладки (фокусу) за замовчуванням.
     - `tabindex="-1"` — прибирає елемент з фокуса при натисканні клавіші Tab, але дозволяє сфокусувати на ньому через JS функцію `focus()`.
     - `tabindex="1"` (та інші додатні значення) — **анти-патерн**, що ламає природну чергу фокусування сторінки. Його варто уникати!

2. **Атрибути WAI-ARIA:** (Web Accessibility Initiative - Accessible Rich Internet Applications). Іноді стандартних тегів замало (наприклад, реалізація "Табів", "Модального вікна" або власного "Select"). Для цього використовуються атрибути `aria-*`, що пояснюють скрінрідерам, як працює ваш кастомний JS компонент:
   - `aria-hidden="true"` — ховає елемент **тільки** від скрінрідера (наприклад, суто декоративну SVG іконку).
   - `aria-expanded="true"` — повідомляє, що випадаюче меню (Акордеон) наразі відкрито.
   - `aria-label="Закрити модальне вікно"` — надає ім'я кнопці `<button>X</button>`, коли візуальна іконка "X" нічого не означає на слух.
   - `role="dialog"` — перетворює `<div>` на "Модальне вікно" в "очах" допоміжних систем.

3. **Контраст кольорів:** Для людей із порушеннями зору чи дальтонізмом контент повинен мати достатній контраст із фоном (за WCAG мінімум 4.5:1 для нормального тексту). Використовуйте плагіни у Chrome, або перевіряйте контраст у Figma (до того як писати код!).

Висновки

Правильна HTML5 архітектура — не привід для творчості, а дотримання строгих стандартів "будівництва". Використовуючи правильні семантичні теги (`<header>`, `<main>`, `<article>`, `<nav>`) замість безликих контейнерів, ви:

1. Робите структуру коду легшою для підтримання (краща читабельність).
2. Істотно допомагаєте "SEO-рушіям", підвищуючи рейтинг у видачі Google (безкоштовний маркетинг).

3. Забезпечуєте інклюзивність (Accessibility), даючи можливість користуватись вашим ресурсом мільйонам людей з інвалідністю за допомогою допоміжних технологій. Тести a11y, і використання атрибутів `alt`, `aria-label` — це база сучасного вебу та стандартів W3C.

Джерела

1. MDN Web Docs: Semantics in HTML (https://developer.mozilla.org/en-US/docs/Glossary/Semantics)
2. Шпаргалка з WAI-ARIA від W3C (https://www.w3.org/WAI/ARIA/apg/)
3. Вступ до Accessibility на MDN (https://developer.mozilla.org/en-US/docs/Learn/Accessibility/What_is_accessibility)

Запитання для самоперевірки

1. Що таке теги в HTML? В чому різниця між тегом та елементом?
2. Яким є головне призначення мета-тегу `<meta name="viewport"...>` (у розділі `<head>`) і що трапиться з мобільним відображенням додання цього тегу?
3. В чому принципова відмінність "Блокових" (block) і "Рядкових" (inline) елементів? Наведіть приклади.
4. Поясніть на прикладах, у яких випадках використовують семантичні теги `<article>` замість `<section>`, і де саме логічно розміщувати `<aside>`?
5. Для чого призначений атрибут `alt` у тегу `<img>`? Які дві проблеми він допомагає вирішити? Назвіть вийняток, у якому `alt` можна лишити зовсім пустим (`alt=""`).
6. Що таке Web Accessibility (a11y) і як працюють інструменти "skrinreder'и" (екранні читалки)?
7. Яка роль атрибута `tabindex` і чому використання `tabindex="1"` чи інших позитивних чисел вважається поганою практикою (анти-патерном) в UI/UX дизайні?
8. Поясніть роботу і застосування атрибутів `aria-hidden` та `aria-label` для кастомних JS віджетів (наприклад, звичайних кнопок-іконок).